

WEBERA SOLUTIONS PK

Game Development Internship

Internship Report

Building an Endless Runner Game in Unity

Prepared by: **Qasim Zahoor**

Role: Game Development Intern

Tool Used: Unity 6 (6000.6.0f1)

1. Introduction

This report documents the work completed during the Game Development Internship at WebEra Solutions PK. The goal of the internship task was to set up a Unity project from scratch and build the foundation of an endless runner game — similar in concept to popular mobile games such as Subway Surfers and Temple Run. The report covers every stage of the process: installing the Unity Editor, creating a new project, configuring the camera and lighting, and building the ground and road for the player to run on.

Project Name	Endless Runner Game
Game Engine	Unity 6 (6000.6.0f1)
Scripting Language	C# (Visual Studio)
Render Pipeline	Universal 3D (URP)
Platform Target	Windows, Mac, Linux

2. Installing the Unity Editor

The first step was to install Unity Hub and, through it, the Unity Editor itself. Unity Hub is the launcher used to manage all Unity projects and editor versions in one place.

2.1 Signing in to Unity Hub

After installing Unity Hub, I signed in with my Unity account. This account is later linked to the project's organization.

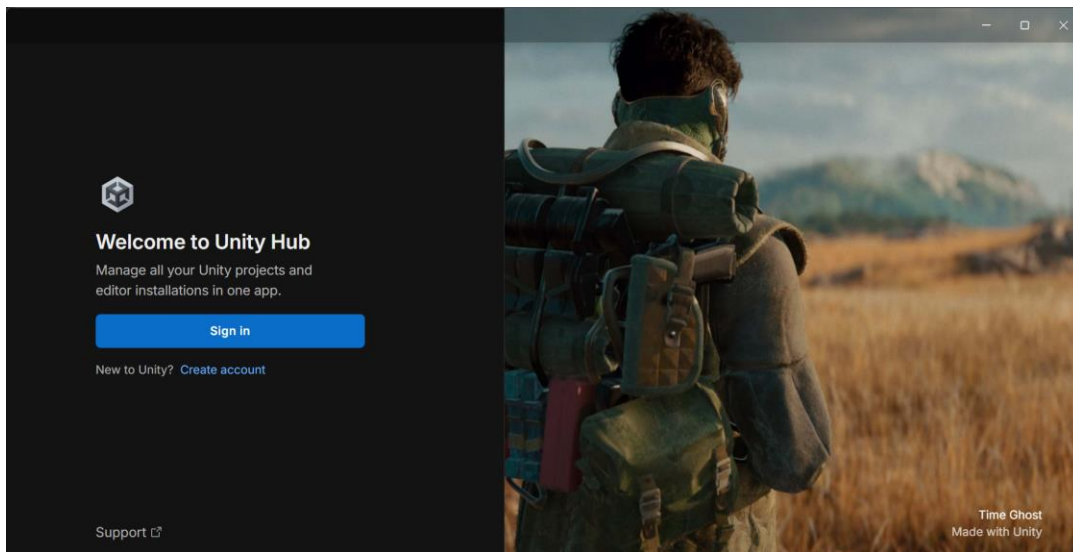


Figure 1: Unity Hub welcome and sign-in screen.

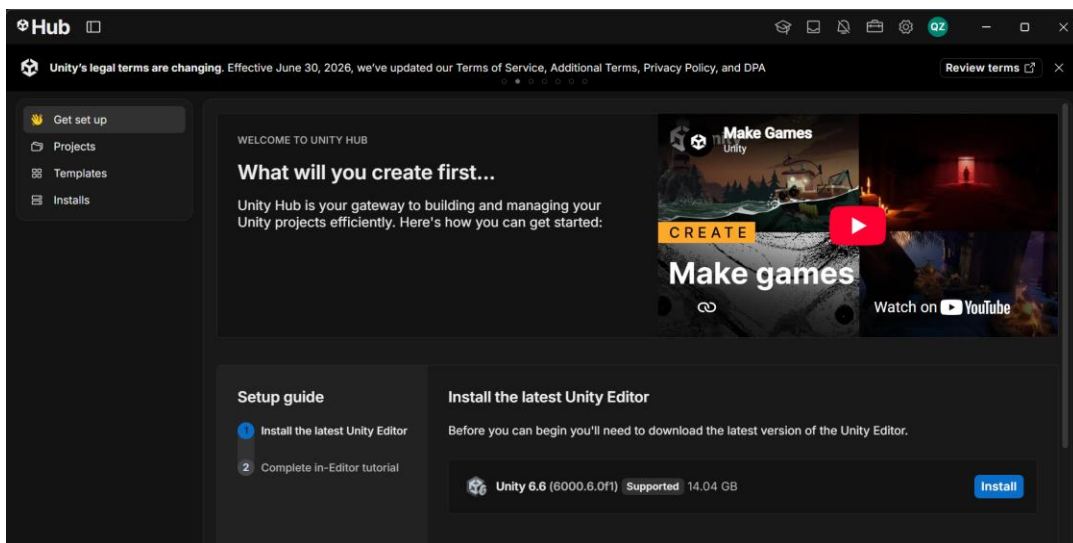


Figure 2: Unity Hub 'Get set up' screen showing the recommended editor version.

2.2 Selecting and Installing the Unity Editor Version

From the Install Editor window, I selected Unity 6.6 (6000.6.0f1), which was marked as the recommended and supported version at the time.

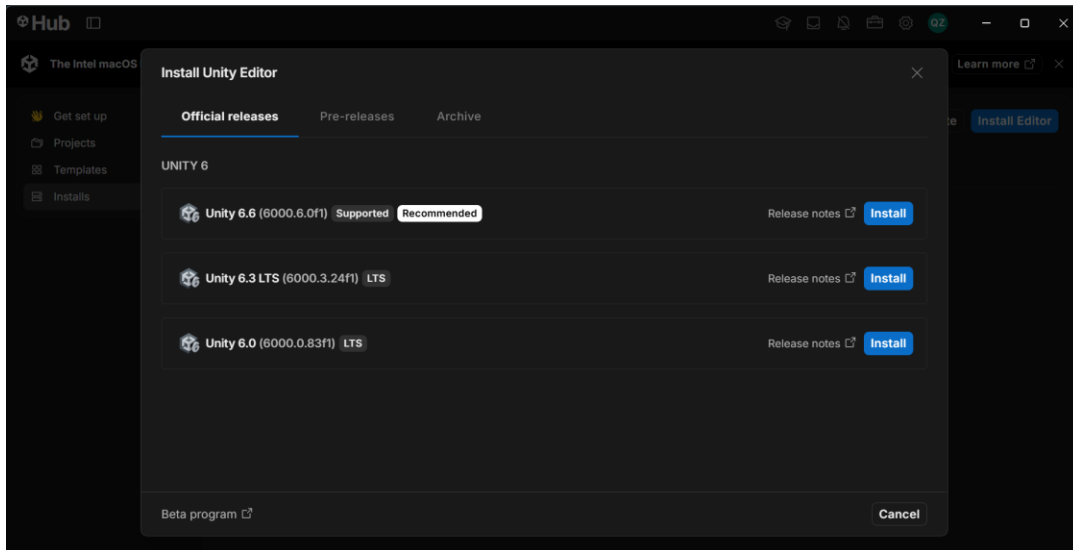


Figure 3: Choosing the Unity 6.6 official release for installation.

2.3 Adding Required Modules

During installation, I selected the additional modules needed for development: Microsoft Visual Studio Community (for writing C# scripts) and Android Build Support, which includes OpenJDK and the Android SDK/NDK tools for future mobile builds.

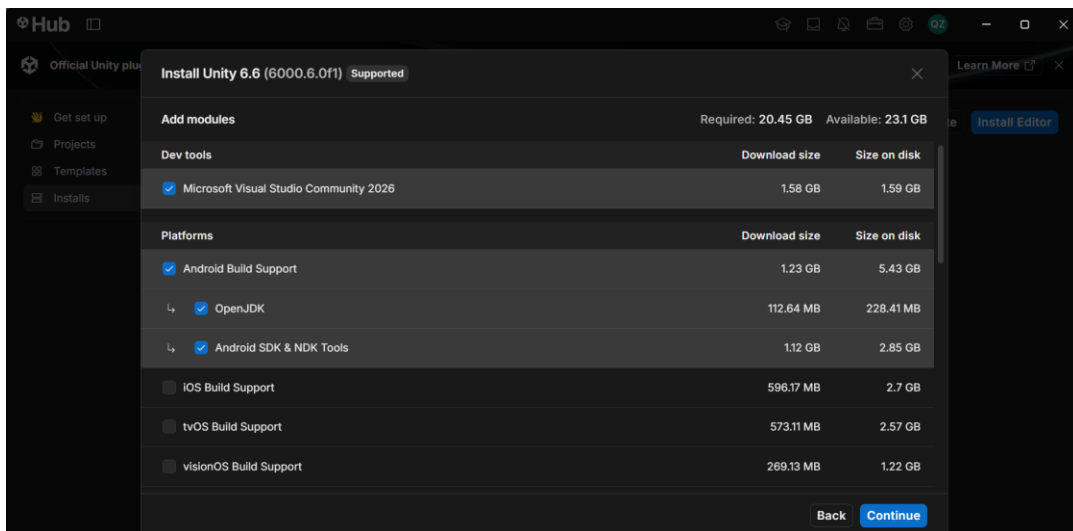


Figure 4: Selecting Visual Studio and Android Build Support modules.

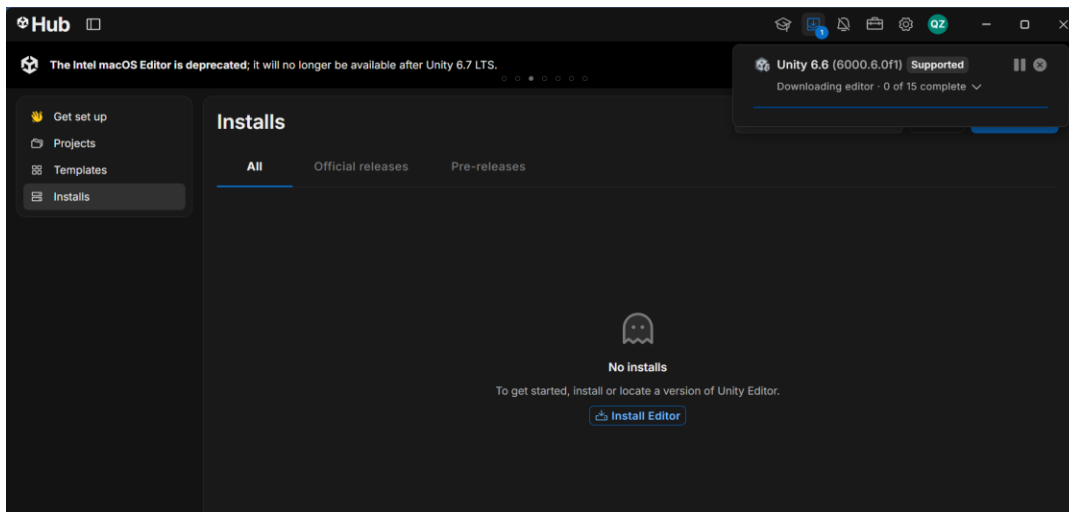


Figure 5: Unity Editor downloading and installing in the background.

3. Creating a New Project

Once the editor was installed, I created a new project using the Universal 3D (URP) template. This template comes pre-configured with the Universal Render Pipeline, which is well suited for a lightweight 3D runner game.

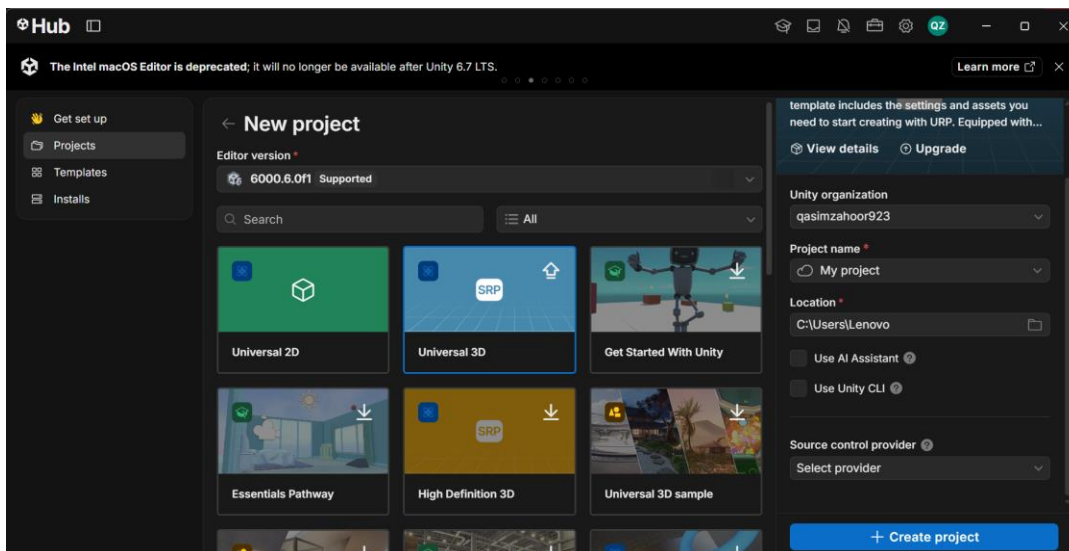


Figure 6: Selecting the Universal 3D template for the new project.

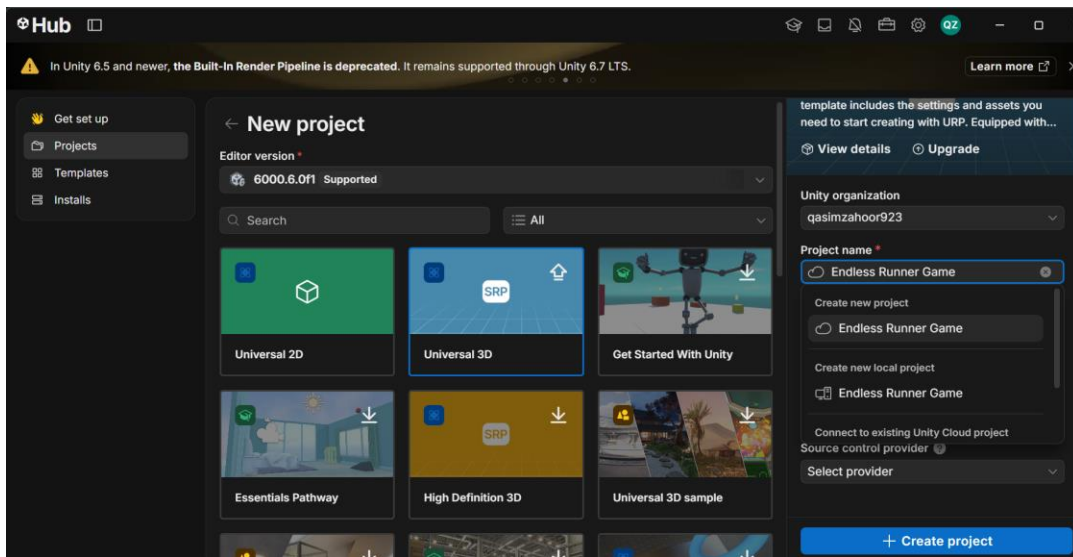


Figure 7: Naming the project “Endless Runner Game” before creation.

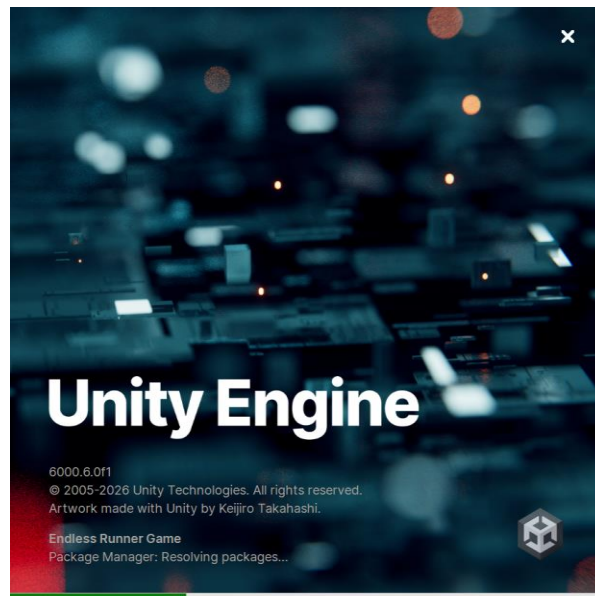


Figure 8: Unity Engine loading screen while the project opens for the first time.

4. Configuring External Tools

To write and edit C# scripts comfortably, I configured Unity's External Tools preferences to use Visual Studio as the external script editor. This was done from Edit → Preferences → External Tools, where I set the External Script Editor option to Visual Studio. This allows scripts — such as those controlling player movement, obstacle spawning, and game logic — to open directly in Visual Studio whenever they are double-clicked inside Unity.

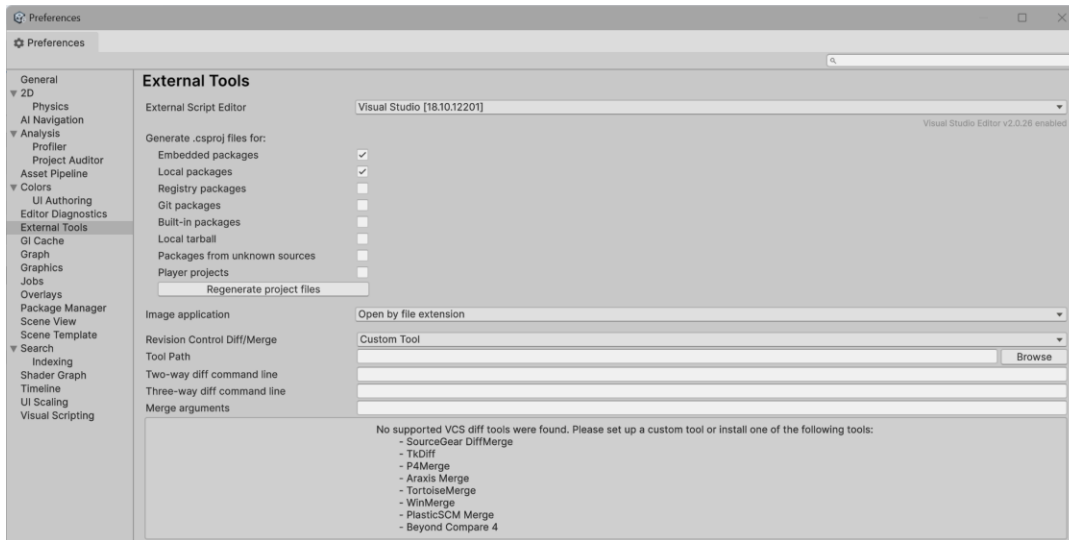


Figure 9: External Tools preferences with Visual Studio set as the script editor.

5. Unity Editor Overview

With the project created, the default scene (SampleScene) opens showing the standard 3D workspace. The Hierarchy panel on the left lists all objects in the scene — initially the Main Camera, Directional Light, and Global Volume — while the Scene view in the center shows the 3D world.

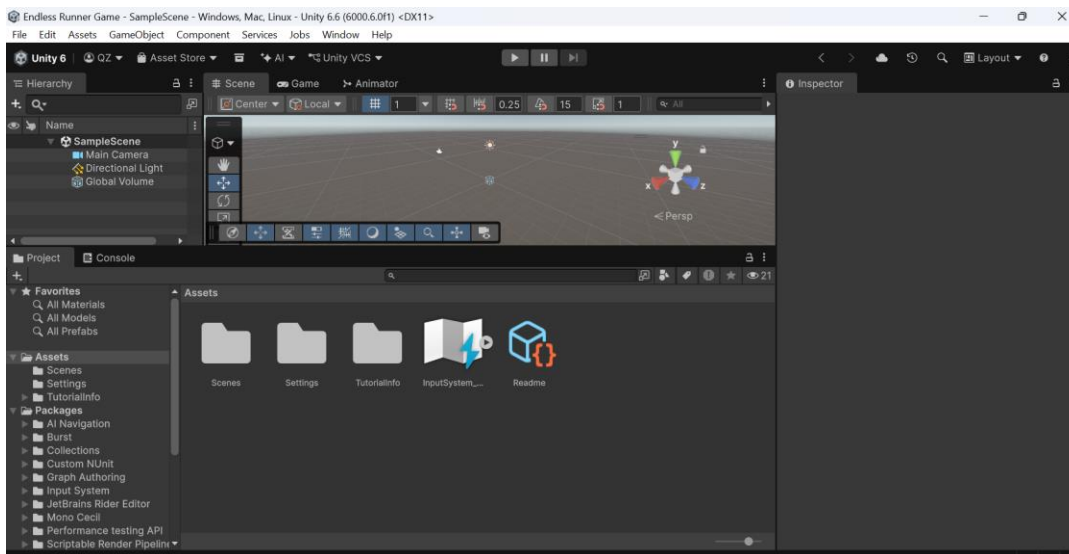


Figure 10: Default Unity 6 editor layout with the empty SampleScene.

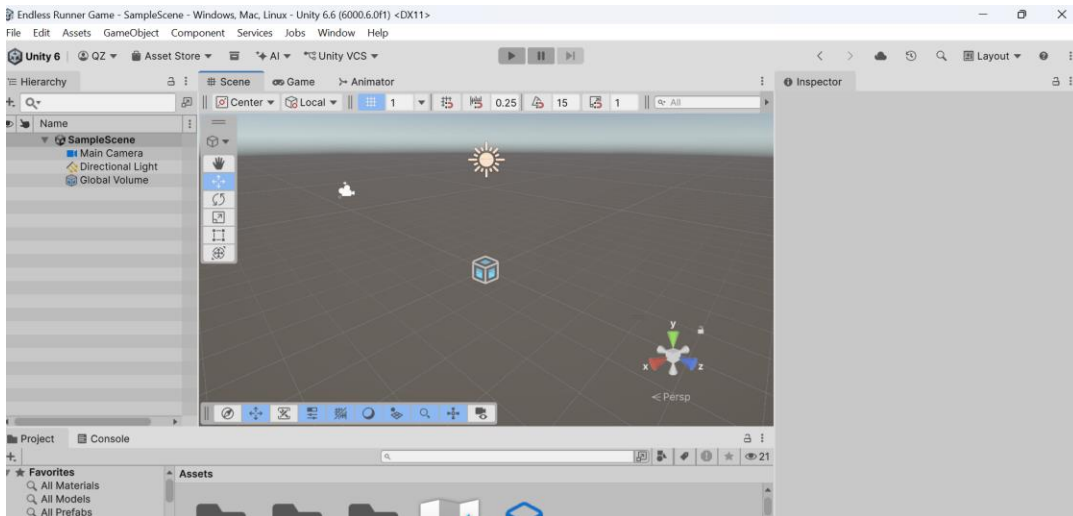


Figure 11: 3D Scene view showing the Main Camera, Directional Light, and world origin.

6. Setting Up the Main Camera

In Unity, the Main Camera acts as the player's eyes, rendering the 3D scene onto the screen. Unlike the Directional Light, both its position and rotation directly affect what the player sees on screen.

6.1 Key Camera Settings

- **Transform (Pos X: 0, Y: 1, Z: -10)** — placed 10 units back on the Z-axis and 1 unit up on the Y-axis, so the camera looks at objects near the world origin (0, 0, 0).
- **Projection (Perspective)** — a standard 3D view in which objects shrink as they move further away, giving natural depth.
- **Field of View (60°)** — controls how wide the camera's view is. A higher value gives a wider view (useful for a sense of speed), while a lower value zooms in.
- **Clipping Planes (Near: 0.3, Far: 1000)** — objects closer than 0.3 units or farther than 1000 units are not rendered.

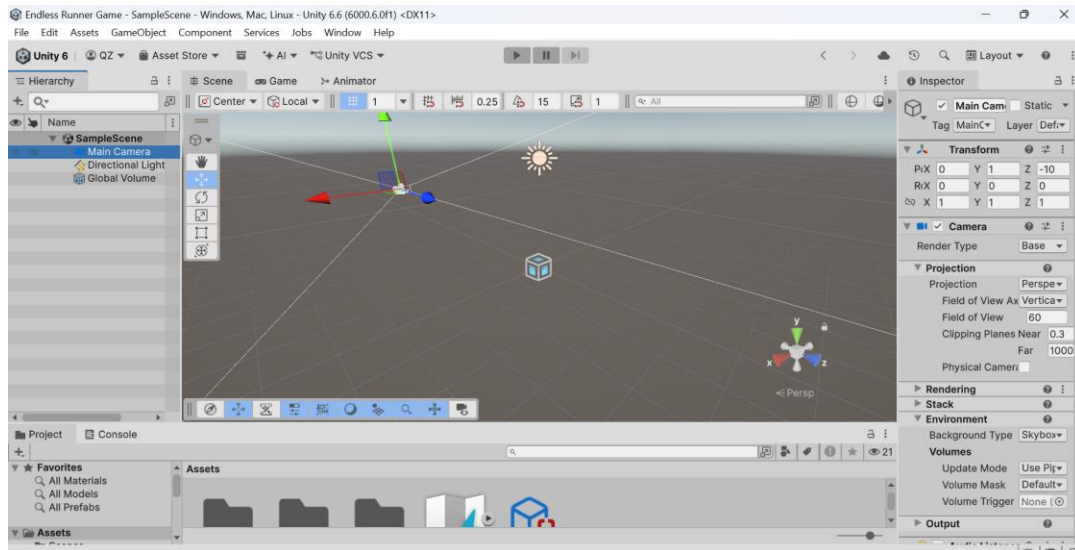


Figure 12: Main Camera Transform and Camera component settings in the Inspector.

6.2 Endless Runner Camera Setup

For a classic third-person endless runner (similar to Subway Surfers), the camera should sit behind and slightly above the player, looking down the running path:

- **Position:** X: 0, Y: 3 to 5, Z: -5 to -8, relative to the player.
- **Rotation:** X: 15° to 25°, tilted slightly downward so the camera can see the path ahead.
- **Scripting:** A smooth-follow C# script attached to the camera keeps it matched to the player's forward (Z-axis) motion without jerking.
- **Cinemachine:** Alternatively, Unity's Cinemachine package can be used with a Virtual Camera set to Body: Transpose or 3rd Person Follow, targeting the player GameObject.

7. Setting Up the Directional Light

A Directional Light in Unity behaves like the sun: it emits parallel light rays across the entire scene from a single direction, no matter where the light object itself is placed.

7.1 Key Concepts

- **Position does not matter:** Moving the light object anywhere in the scene (for example X: 0, Y: 3, Z: 0) does not change the lighting or the shadows at all.
- **Only rotation matters:** The rotation of the light (Rotation X, Y, Z) is the only factor that determines where shadows fall and how highlights appear.
- **Infinite distance:** Unity treats the light source as if it is infinitely far away, so it lights the entire world evenly.

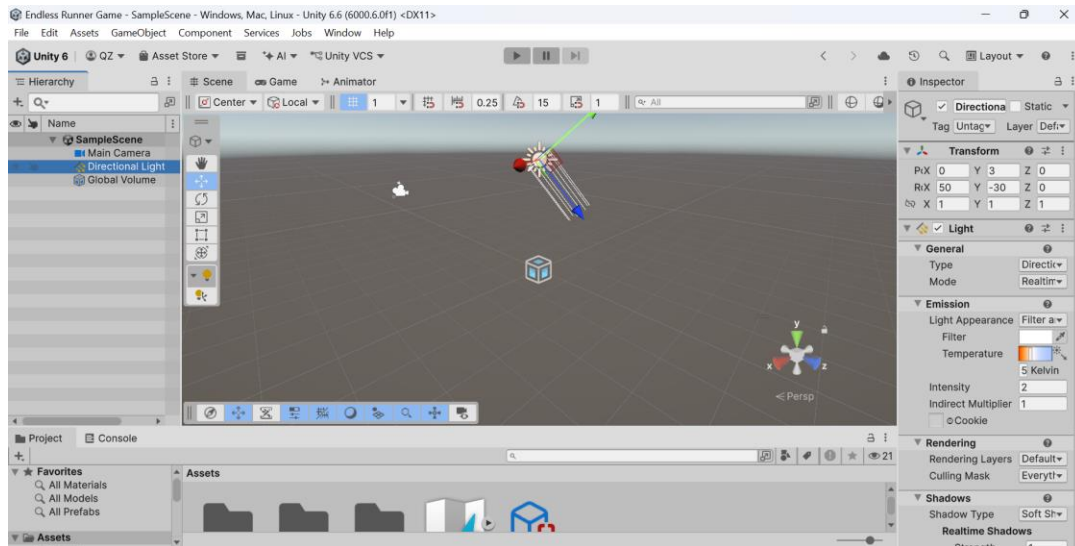


Figure 13: Directional Light Inspector showing rotation and intensity settings.

7.2 Lighting the Endless Runner Scene

For this project, the Directional Light was rotated to X: 50°, Y: -30°, Z: 0°:

- **Sunlight angle:** The light comes down from above at a natural slanting angle, while the Y rotation of -30° adds light slightly from the side, producing clear shadows behind and beside the player, coins, and obstacles.
- **Infinite track coverage:** As the player runs forward (from Z = 0 to Z = 10,000 and beyond), the Directional Light does not need to move. Because it lights the entire world evenly, every newly spawned obstacle and ground tile is lit automatically.

Key inspector settings used for the light:

- **Intensity:** Set to 2, controlling the overall brightness of the sunlight.
- **Shadow Type:** Set to Soft Shadows, which slightly blurs shadow edges for a cleaner, higher-quality look.
- **Mode:** Set to Realtime, so lighting is recalculated every frame — allowing moving objects such as the running player to cast accurate, moving shadows.

8. Building the Ground and Road

With the camera and lighting configured, the next step was to build the physical running track.

8.1 Creating the Ground

From the Hierarchy panel, I chose GameObject → 3D Object → Plane and renamed the new object to “Ground.” This plane acts as the base floor beneath the running track.

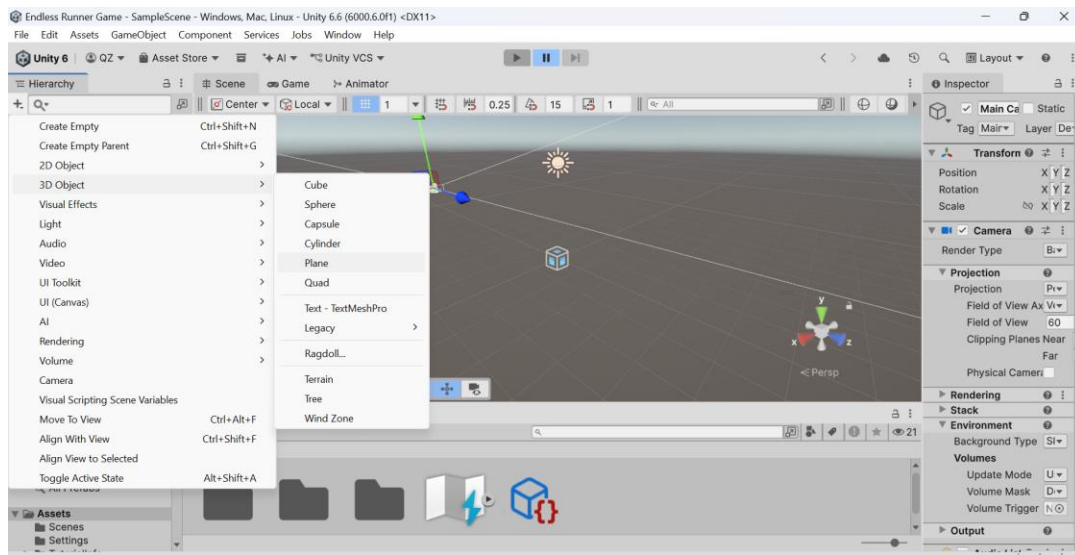


Figure 14: Creating a 3D Plane object from the GameObject menu.

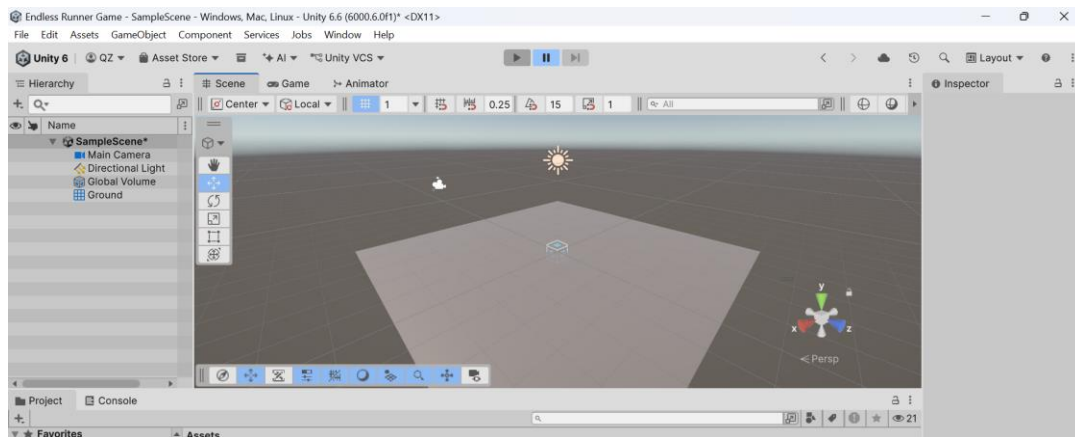


Figure 15: The Plane added to the scene and renamed "Ground" in the Hierarchy.

8.2 Creating the Road

Next, I chose GameObject → 3D Object → Cube and renamed it to "Road." This cube represents the running path itself, sitting above the ground plane. It was scaled and positioned as follows:

Scale X	8
Scale Y	0.1
Scale Z	30
Position X	0
Position Y	0
Position Z	15

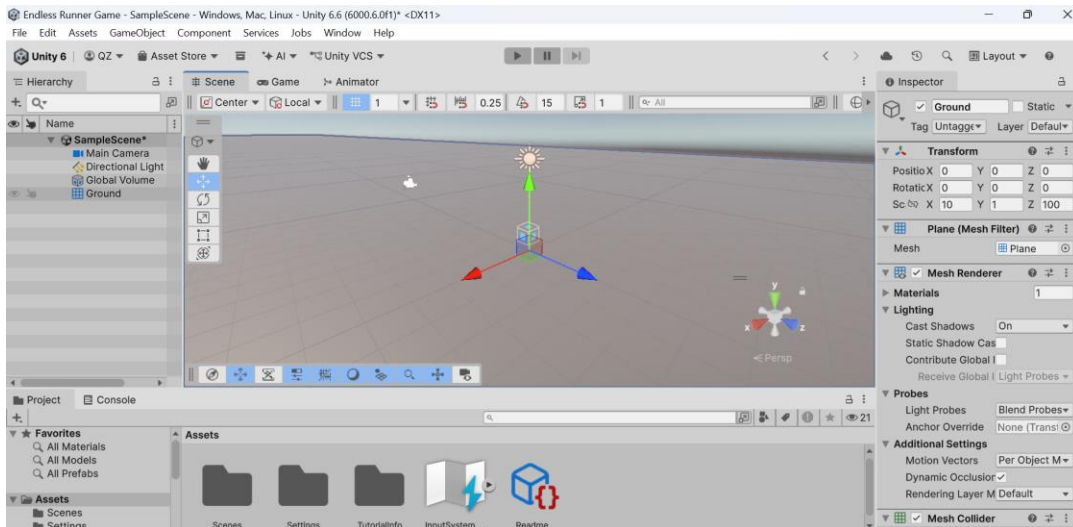


Figure 16: The Ground object selected, showing its Transform and Mesh Renderer settings.

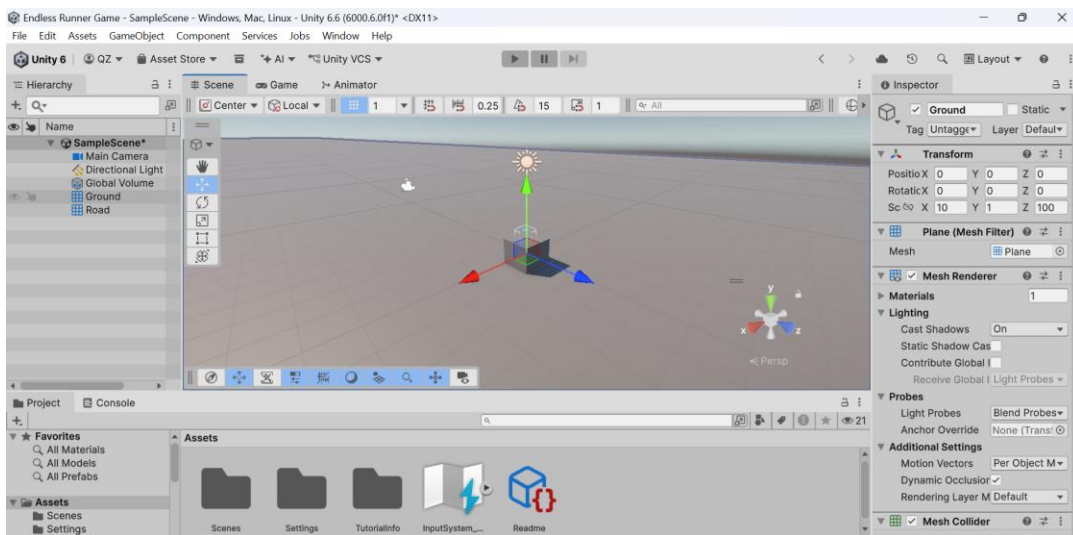


Figure 17: The Road cube added to the Hierarchy, above the Ground object.

9. Final Result

After applying the scale and position values above, the Road sits correctly on top of the Ground, forming a long, narrow running path that stretches into the distance — the foundation for the endless runner track.

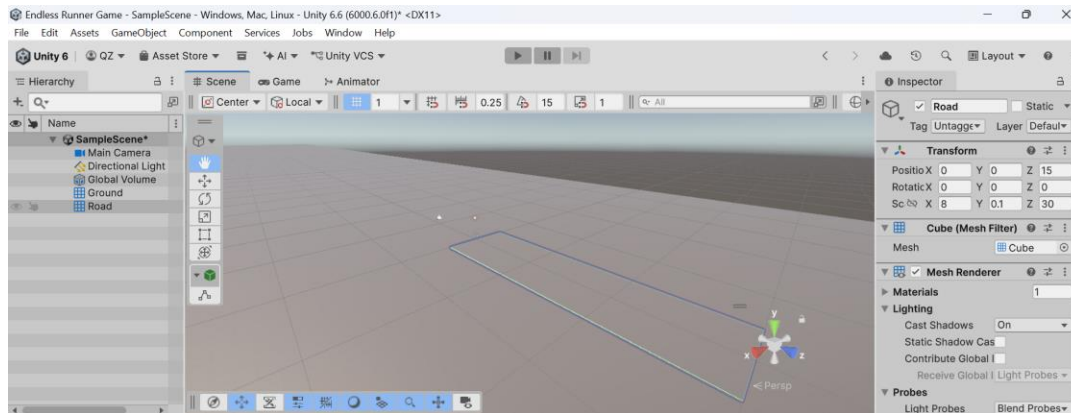
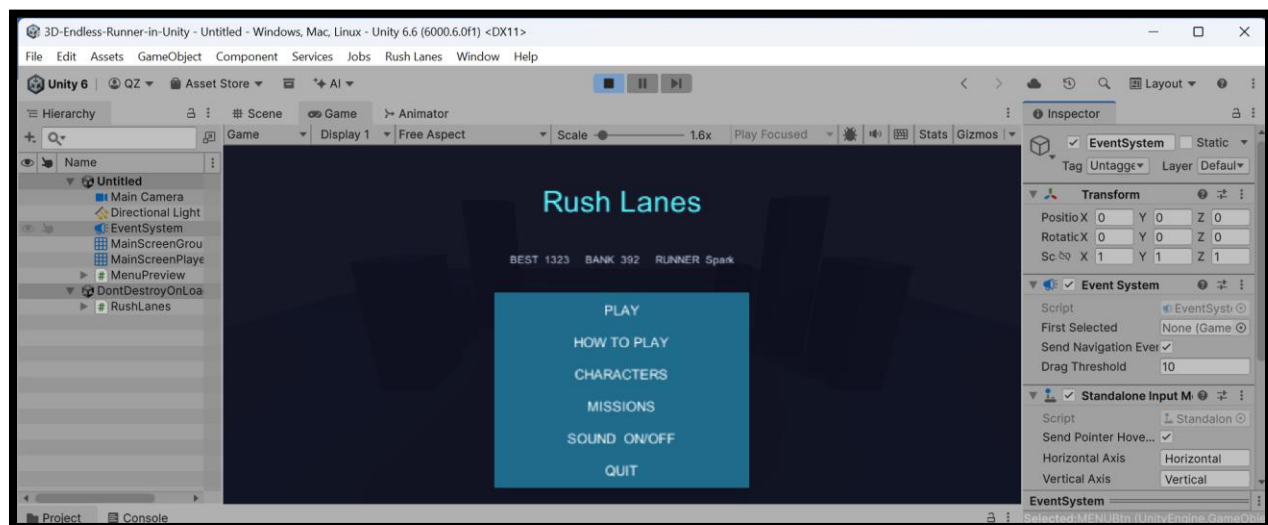


Figure 18: Final result — Ground and Road correctly positioned in the scene.

10. Conclusion

By the end of this task, a working Unity project for the Endless Runner Game had been set up with a properly configured camera, lighting, and the basic ground and road layout needed for gameplay. This foundation is now ready for the next stages of development, including player movement scripting, obstacle spawning, and scoring.

11. Event System:



PART B

Rush Lanes — Main Screen, Scene Objects, Menu System and Scripts

Continuation of the Endless Runner internship report (Sections 11 – 20)

Part A of this report covered the installation of Unity, the creation of the project, and the basic camera, lighting, ground and road setup. Part B documents the next stage of the internship task: the complete main screen of the finished game “Rush Lanes”, every object that lives in the scene, the menu that the player sees when the game starts, and the C# scripts that drive the audio and the character system. Each screenshot taken from the Unity Editor is explained step by step, together with the exact values that were entered in the Inspector.

11. Rush Lanes — Project Overview

After the basic track was working, the project was extended into a complete playable game named Rush Lanes. The table below summarises the final technical setup of the project.

Item	Detail
Game Title	Rush Lanes (3D Endless Runner)
Unity Version	Unity 6 — 6000.6.0f1
Render Pipeline	Built-in Render Pipeline
Color Space	Gamma
Scripting Language	C# (Visual Studio 2026)
Namespace	RushLanes
Total Scripts	21 runtime scripts + 1 editor script
Scene Files	Assets/Scenes/MainScene.unity
Repository	https://github.com/qasimzahoor825/Rush-Lanes

11.1 Folder Structure

All source files are organised inside the Assets folder by responsibility, so that every system can be located quickly:

```

Assets/
├── Editor/ProjectConfigurator.cs      → 1-click project settings tool
├── Scenes/MainScene.unity            → shell scene (camera + light)
├── Scripts/
│   ├── Audio/AudioManager.cs        → procedural music + SFX
│   ├── Characters/CharacterCatalog.cs → 3 runners (Spark / Ember / Nova)
│   ├── Core/
│   │   ├── GameBootstrap, GameFlow, GameIds,
│   │   ├── RushLanesSceneBuilder, MaterialFactory,
│   │   └── MainScreenPlayerSetup
│   ├── Data/SaveSystem.cs           → PlayerPrefs JSON save
│   ├── Missions/MissionManager.cs    → 3 missions
│   ├── Player/
│   │   ├── PlayerController, PlayerVisual, SwipeInput
│   ├── UI/
│   │   ├── GameUI, UiFactory         → 100% code-generated UI
│   └── World/
│       ├── GroundTile, TrackSpawner, CameraRig,
│       └── CoinPickup, ObstacleHit, PowerUpPickup

```

11.2 Role of Each Folder

Folder	Responsibility
Editor	ProjectConfigurator.cs adds a custom menu item that applies all required Project Settings (tags, layers, physics, quality) in one click, so the project never has to be configured by hand again.
Audio	AudioManager.cs generates every sound effect and the background music bed from mathematics at runtime — no .wav or .mp3 files are imported.
Characters	CharacterCatalog.cs holds the definition of the three playable runners and the coin cost required to unlock them.
Core	GameBootstrap starts the game, GameFlow controls the state machine (menu → play → game over), RushLanesSceneBuilder creates the scene objects, and MaterialFactory produces the materials used by every mesh.
Data	SaveSystem.cs serialises the player's progress to JSON and stores it in PlayerPrefs.
Missions	MissionManager.cs tracks the three daily missions and awards the coin reward.
Player	PlayerController handles lane movement, jumping and sliding; PlayerVisual colours the runner; SwipeInput reads touch and mouse gestures.
UI	UiFactory builds every Canvas, panel and button through code; GameUI updates the score, coin and mission labels each frame.
World	GroundTile, TrackSpawner, CoinPickup, ObstacleHit, PowerUpPickup and CameraRig together form the endless track and the camera that follows the runner.

11.3 Why the Scene File Is Almost Empty

MainScene.unity deliberately contains only a Main Camera and a Directional Light. Everything else — the ground, the runner preview, the menu and the managers — is created in code the moment the game starts. This approach was chosen for three reasons:

- **No broken references** — because no object is stored in the scene file, nothing can lose its link when the project is moved to another computer or pushed to GitHub.
- **Smaller repository** — the scene file stays a few kilobytes instead of several megabytes.

- **Easier tuning** — lane width, speed, colours and spawn distances are all constants inside scripts, so a single number change rebuilds the whole level.

The build order is: GameBootstrap → SaveSystem.Load() → MaterialFactory → RushLanesSceneBuilder → UiFactory → GameFlow. The screenshots in the following sections were taken in Play Mode, which is why the objects appear under the root object “Untitled” in the Hierarchy — they exist only while the game is running.

The same objects can also be created by hand from the GameObject menu. The step-by-step instructions given in each section below describe exactly that manual procedure, so the scene can be rebuilt without a single line of code if required.

12. Event System

The Event System is the object that allows Unity's UI to respond to input. Without it, the menu buttons are drawn on screen but nothing happens when they are clicked. It listens to the keyboard, the mouse and the touch screen, decides which UI element is under the pointer, and sends the click event to that element.

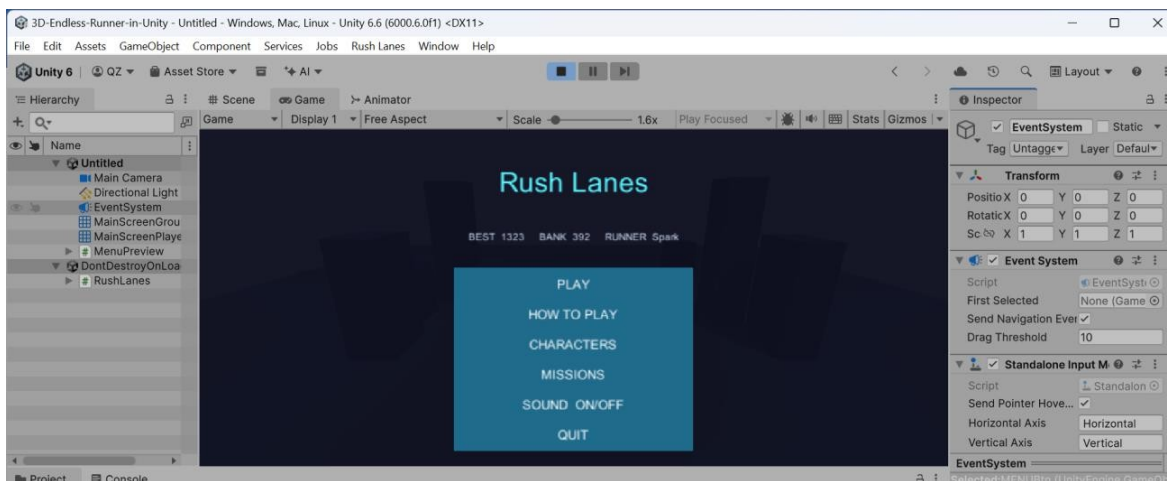


Figure 19: The EventSystem object selected in the Hierarchy with its Inspector components.

12.1 What the Screenshot Shows

On the left, the Hierarchy lists the objects that make up the running game: Main Camera, Directional Light, EventSystem, MainScreenGround, MainScreenPlayer, MenuPreview, and — under DontDestroyOnLoad — the RushLanes manager object. In the centre, the Game view shows the finished main menu with the title “Rush Lanes”, the statistics row (BEST 1323, BANK 392, RUNNER Spark) and the six menu buttons. On the right, the Inspector shows the two components attached to the EventSystem object.

Component / Field	Value Used	What It Does
Transform	Pos 0, 0, 0 · Rot 0, 0, 0 · Scale 1, 1, 1	The Event System is not a visible object, so its position in the world does not matter. It is left at the origin.
Event System	Script: EventSystem	The main component. It runs once per frame, collects input from every input module, and raises the click, drag and select events for the UI.
First Selected	None (GameObject)	Left empty because the menu is operated with the

Component / Field	Value Used	What It Does
		mouse and touch, not with keyboard navigation. If a button were assigned here, it would be highlighted automatically on start-up.
Send Navigation Events	Enabled (ticked)	Allows the arrow keys and the WASD keys to move the selection between buttons — useful when the game is tested on a keyboard.
Drag Threshold	10	The pointer must move more than 10 pixels before the input is treated as a drag instead of a click. This prevents a small shake of the finger from cancelling a button press.
Standalone Input Module	Horizontal / Vertical axes	Converts the raw mouse and keyboard input into UI events. The Horizontal and Vertical axis names match the ones defined in the Input Manager.
Send Pointer Hover To Parent	Enabled	Hover events also travel up to the parent object, which lets a whole button panel react when a child label is hovered.

12.2 How the Event System Was Created (Step by Step)

1. In the Hierarchy panel, right-click on empty space and choose **GameObject → UI → Event System**. Unity creates a new object called **EventSystem**.
2. Select the new object and confirm in the Inspector that two components are present: **Event System** and **Standalone Input Module**.
3. Leave **First Selected** empty so that no button is highlighted when the menu opens.
4. Tick **Send Navigation Events** so that the menu can also be operated with the keyboard.
5. Set **Drag Threshold** to 10 — the default value works well for both mouse and touch.
6. Leave the **Transform** at 0, 0, 0. The object is invisible and its position has no effect.

Note: when a Canvas is created first (**GameObject → UI → Canvas**), Unity adds an **Event System** automatically. In this project the Canvas is built by **UiFactory** at runtime, so the **Event System** is created together with it by the same script.

13. Main Screen Ground

The main menu is not shown against a flat colour. Behind the menu panel there is a real 3D scene — a dark road with blurred buildings on both sides — so that the menu already looks like part of the game world. **MainScreenGround** is the floor of that background scene.

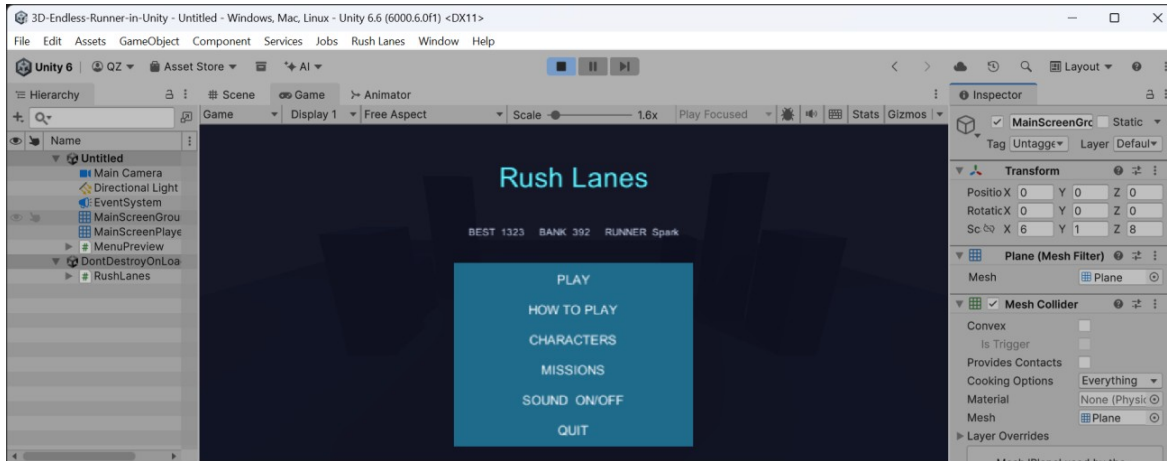


Figure 20: MainScreenGround selected, showing its Transform, Mesh Filter and Mesh Collider.

13.1 Inspector Values

Property	Value Used	Reason
Name	MainScreenGround	A clear name so the object can be told apart from the ground tiles used during gameplay.
Tag / Layer	Untagged / Default	No tag is needed because no script searches for this object; it is purely decorative.
Position	X 0 · Y 0 · Z 0	Placed at the world origin, directly in front of the menu camera.
Rotation	X 0 · Y 0 · Z 0	A Plane in Unity already faces upwards, so no rotation is required.
Scale	X 6 · Y 1 · Z 8	A Unity Plane is 10 × 10 units by default, so this scale produces a floor of 60 units wide and 80 units long — wide enough that the edges never appear on screen.
Mesh Filter	Mesh: Plane	Defines the shape. A Plane is used instead of a Cube because it is a single flat quad and therefore the cheapest possible floor.
Mesh Collider	Convex: off · Mesh: Plane	Added so that the preview runner stands on the floor instead of falling through it. Convex is left off because a static flat surface does not need a convex hull.
Cooking Options	Everything	Unity's default. It cleans the mesh before building the collision data, which avoids warnings for thin meshes.
Material	Dark road material from MaterialFactory	A dark grey, low-gloss material that keeps the background quiet so the menu text stays readable.

13.2 How the Ground Was Created (Step by Step)

1. From the top menu choose GameObject → 3D Object → Plane.

2. Rename the new object to MainScreenGround in the Hierarchy (F2 or slow double-click).
3. In the Inspector, set Position to (0, 0, 0) and Rotation to (0, 0, 0).
4. Set Scale to X = 6, Y = 1, Z = 8 so the floor covers the whole camera view.
5. Check that a Mesh Collider is already attached — Unity adds one to a Plane automatically. Leave Convex unticked.
6. Drag the dark road material onto the object in the Scene view, or assign it in the Mesh Renderer → Materials slot.
7. Press Play and confirm that the floor fills the lower half of the Game view and that no edge is visible at the sides.

14. Main Screen Player

MainScreenPlayer is the runner that stands on the road behind the menu. It is the same capsule shape that the player controls during gameplay, so the character selected in the Characters screen can be previewed before the game is even started.

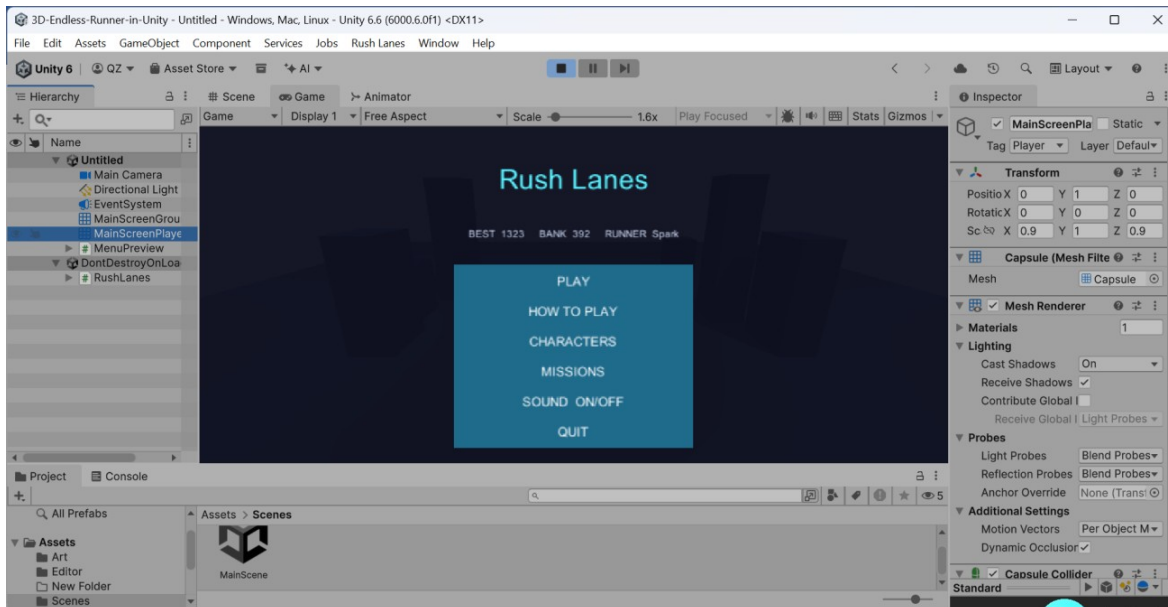


Figure 21: MainScreenPlayer selected, showing the Player tag, the Capsule mesh and the Capsule Collider.

14.1 Inspector Values

Property	Value Used	Reason
Name	MainScreenPlayer	Identifies the menu preview runner.
Tag	Player	The Player tag is used so that the same colouring script (PlayerVisual) can find this object and apply the colours of the currently selected character.
Position	X 0 · Y 1 · Z 0	Y = 1 lifts the capsule so that its lower half sits exactly on the ground plane instead of sinking into it. A default Unity capsule is 2 units tall, so half of its height is 1 unit.

Property	Value Used	Reason
Rotation	X 0 · Y 0 · Z 0	The runner faces the camera straight on.
Scale	X 0.9 · Y 1 · Z 0.9	Slightly slimmer than a default capsule, which makes the character look more like a person and less like a pill.
Mesh Filter	Mesh: Capsule	A capsule is used as the runner body because it is rounded at the bottom and therefore slides smoothly over small bumps in the track.
Mesh Renderer	Cast Shadows: On · Receive Shadows: On	The runner casts a shadow on the road, which adds depth to the menu background.
Capsule Collider	Default height 2, radius 0.5	Keeps the runner standing on the Mesh Collider of the ground and is later reused for obstacle and coin collision during gameplay.
Material	Character body colour (Spark by default)	Assigned at runtime from CharacterCatalog.Selected, so the preview always matches the character the player has chosen.

14.2 How the Player Preview Was Created (Step by Step)

1. Choose GameObject → 3D Object → Capsule and rename it to MainScreenPlayer.
2. Open the Tag dropdown at the top of the Inspector and select Player. (If the tag does not exist, choose Add Tag..., create "Player", then assign it.)
3. Set Position to (0, 1, 0) so the capsule rests on the ground plane.
4. Set Scale to (0.9, 1, 0.9) to slim the body slightly.
5. Leave the Capsule Collider that Unity attached automatically; it is required for the runner to stand on the floor.
6. In Mesh Renderer → Lighting, confirm that Cast Shadows is set to On and Receive Shadows is ticked.
7. Assign the character material. During gameplay this is done by MainScreenPlayerSetup, which reads the selected character and applies its body and accent colours.

15. Menu Preview

MenuPreview is an empty GameObject that carries the Ground Tile script. Its job is to make the background of the menu feel alive: the same tile-scrolling logic that is used during gameplay is applied to the menu scene, so the road appears to move slowly under the runner even before the player presses PLAY.

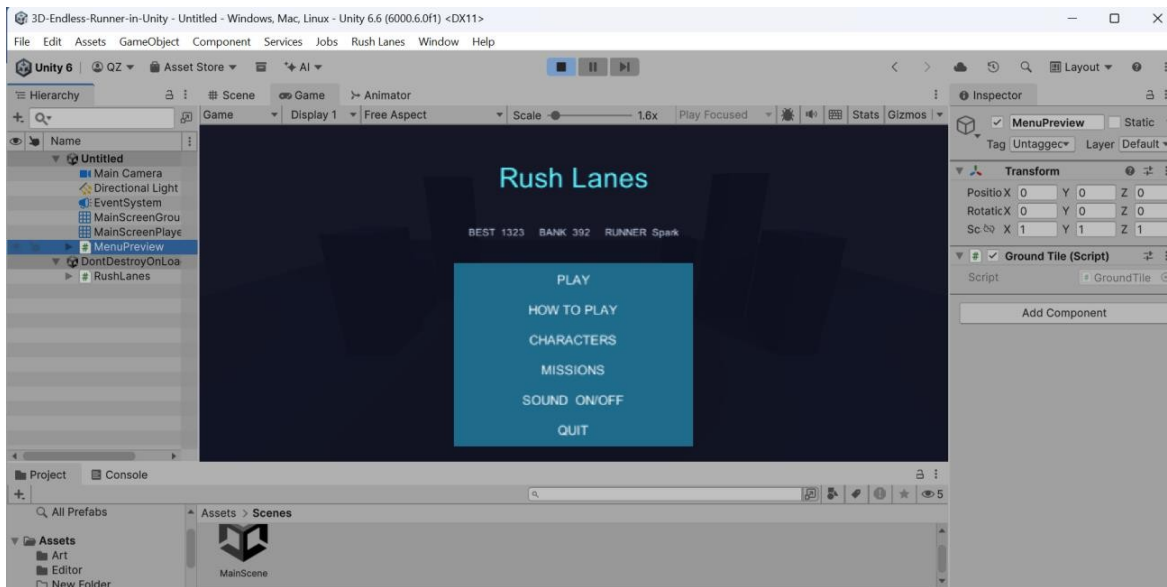


Figure 22: MenuPreview selected, showing the Ground Tile (Script) component.

15.1 Inspector Values

Property	Value Used	Reason
Name	MenuPreview	Holds the animated part of the menu background.
Position / Rotation / Scale	0, 0, 0 • 0, 0, 0 • 1, 1, 1	An empty container object. Its children are positioned relative to it, so the parent itself stays at the default transform.
Ground Tile (Script)	GroundTile.cs	The only component on the object. It moves the tile towards the camera each frame and recycles it back to the start of the track once it has passed behind the camera, producing an endless scrolling road.
Children	Road tile and side buildings	Created by RushLanesSceneBuilder and parented to MenuPreview so that the whole background can be enabled or disabled with one line.

15.2 How the Menu Preview Was Created (Step by Step)

1. Choose GameObject → Create Empty and rename it to MenuPreview.
2. Leave the Transform at Position (0, 0, 0), Rotation (0, 0, 0) and Scale (1, 1, 1).
3. Click Add Component, search for Ground Tile and attach the GroundTile script.
4. Drag the road tile and the blurred building objects into MenuPreview so they become its children.
5. Press Play. The tiles should now scroll slowly towards the camera and loop back without any visible seam.

16. RushLanes — The Manager Object

RushLanes is the most important object in the game. It is created by GameBootstrap at start-up, marked with DontDestroyOnLoad, and therefore survives every scene change. All the managers that must never

be destroyed live on this single object: the audio manager, the two audio sources, the UI controller and the game-flow state machine.

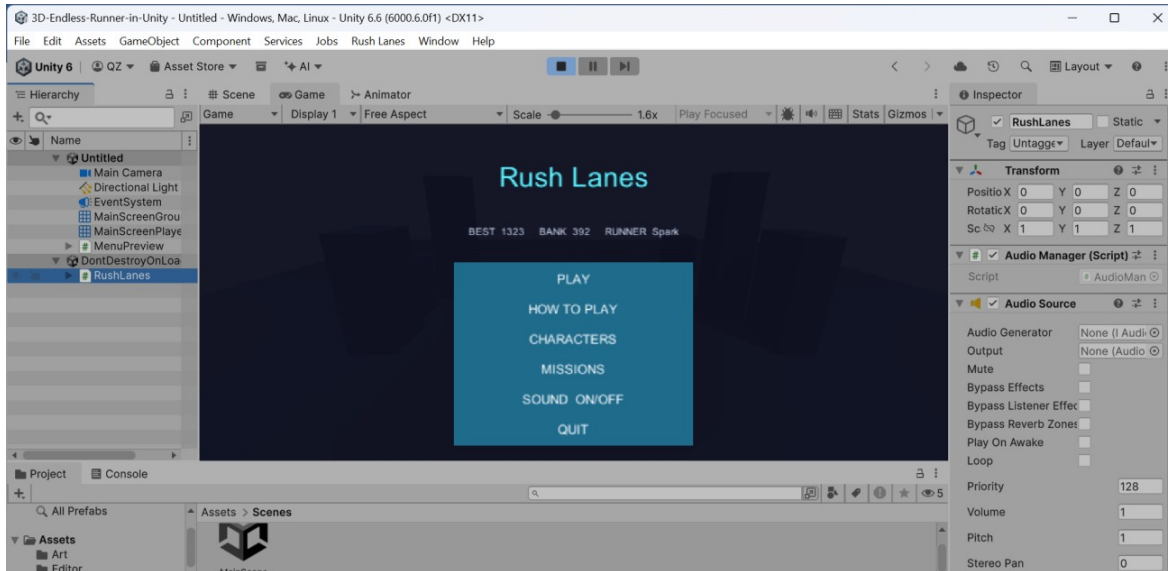


Figure 23: RushLanes under DontDestroyOnLoad, showing the Audio Manager script and the first Audio Source.

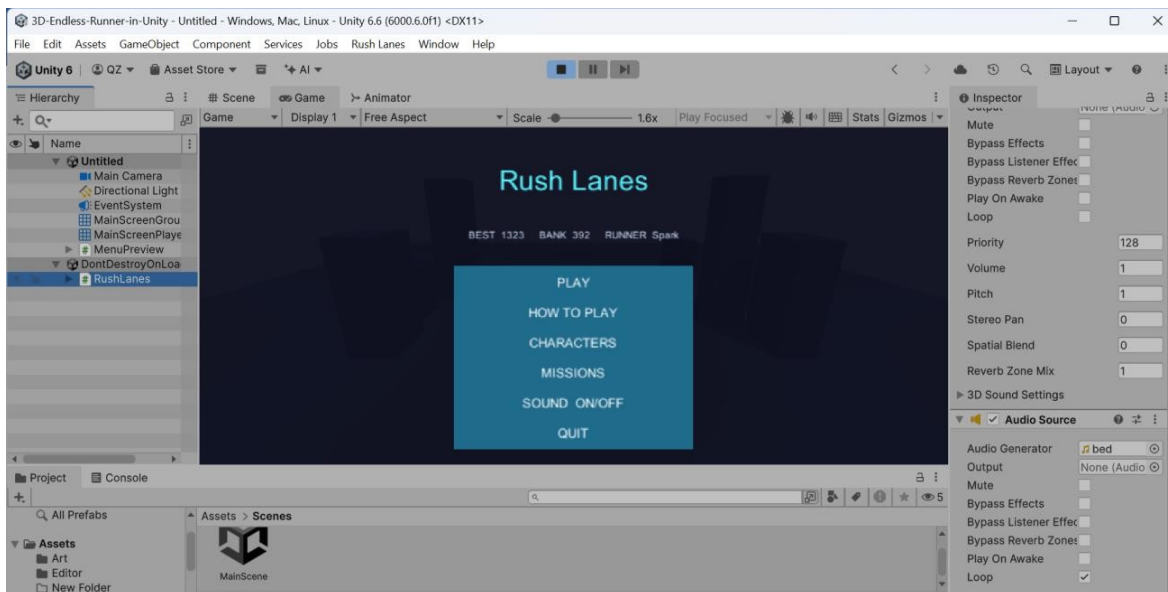


Figure 24: The second Audio Source of RushLanes — the looping music channel.

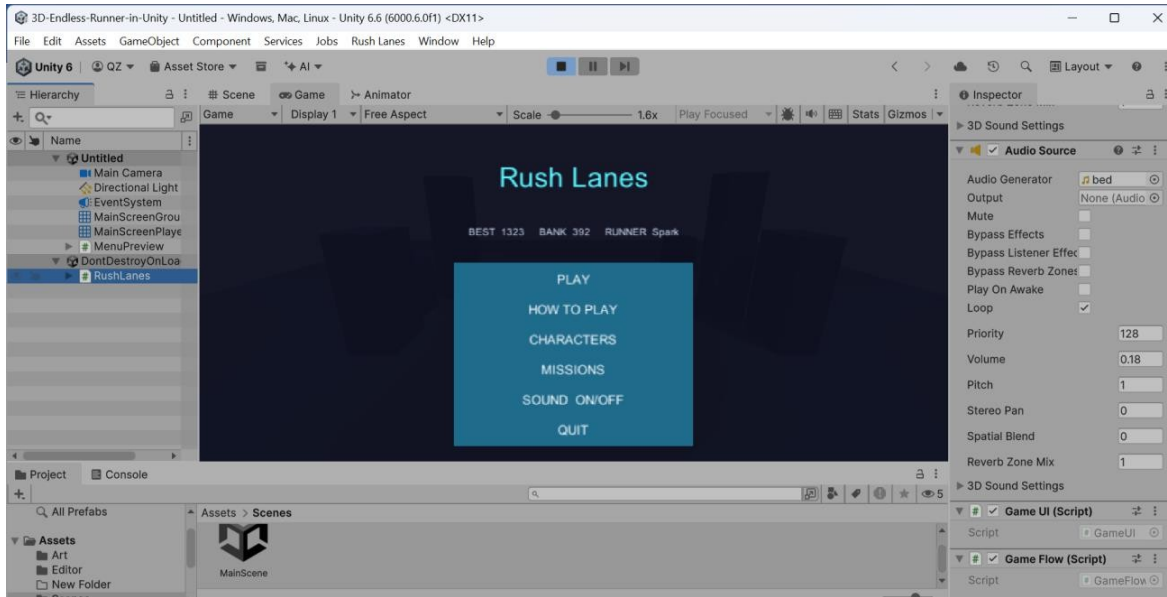


Figure 25: The music Audio Source settings together with the Game UI and Game Flow scripts.

16.1 Components Attached to RushLanes

Component	Key Settings	Purpose
Audio Manager (Script)	AudioManager.cs	Creates every sound effect mathematically at start-up and exposes simple methods such as Coin(), Jump() and GameOver() that the rest of the game can call.
Audio Source #1 (SFX)	Play On Awake: off · Loop: off	The one-shot channel. Every sound effect is played through PlayOneShot on this source, so several effects can overlap without cutting each other off.
Audio Source #2 (Music)	Audio Generator: bed · Loop: on · Volume 0.18 · Play On Awake: off · Priority 128	The background music channel. The clip named “bed” is the four-second procedural loop generated by the LoopBed() method. The low volume of 0.18 keeps the music behind the sound effects.
Spatial Blend	0 (2D)	Both sources are fully 2D so the volume never changes with the distance between the runner and the camera.
Game UI (Script)	GameUI.cs	Updates the score, distance, coin counter, mission text and the game-over panel every frame.
Game Flow (Script)	GameFlow.cs	The state machine that moves the game between Splash, Menu, Tutorial, Play, Pause and Game Over.

16.2 Why DontDestroyOnLoad Is Used

When a scene is reloaded after the runner dies, Unity destroys every object in that scene. If the audio manager were an ordinary scene object, the music would stop and every sound clip would have to be generated again — which causes a visible freeze. By calling DontDestroyOnLoad on RushLanes, the object

is moved into a special scene that is never unloaded, so the music keeps playing seamlessly from the menu into the game and back again, and the saved data held by the managers is never lost.

16.3 How the Manager Object Was Created (Step by Step)

1. Choose GameObject → Create Empty and rename it to RushLanes.
2. Click Add Component and attach AudioManager, GameUI and GameFlow.
3. Add two Audio Source components (Add Component → Audio → Audio Source).
4. On the first Audio Source, untick Play On Awake and untick Loop. This is the sound-effect channel.
5. On the second Audio Source, untick Play On Awake, tick Loop, and set Volume to 0.18. This is the music channel.
6. Set Spatial Blend to 0 (2D) on both sources so the sound is not affected by distance.
7. In the script that starts the game, call DontDestroyOnLoad(gameObject) so that the object survives every scene load.

17. Building the Main Menu Screen

The menu shown in every screenshot is the first thing the player sees. It is built out of three parts: the game title at the top, a statistics row underneath it, and a vertical panel of six buttons in the centre of the screen. In the finished project all of these elements are generated by UiFactory, but they can be recreated by hand with the steps given below.

17.1 Layout of the Menu

Element	Content	Design Note
Title	“Rush Lanes” — large cyan text, top centre	The cyan colour (approximately R 0.15, G 0.92, B 0.95) matches the body colour of the default character, Spark, so the branding stays consistent.
Statistics row	BEST 1323 · BANK 392 · RUNNER Spark	Shows the best score achieved, the number of coins currently banked, and the name of the selected character. All three values are read from SaveSystem at start-up.
Button panel	Teal panel with six buttons	A solid panel is placed behind the buttons so the white text stays readable against the moving 3D background.
Buttons	PLAY · HOW TO PLAY · CHARACTERS · MISSIONS · SOUND ON/OFF · QUIT	Stacked vertically and spaced evenly. A Vertical Layout Group keeps them aligned regardless of the screen resolution.

17.2 What Each Button Does

Button	Action Performed
PLAY	Calls GameFlow.StartRun(). The menu fades out, the track spawner builds the first tiles, and the runner starts moving forward. On the very first launch the

Button	Action Performed
	tutorial screen is shown once before the run begins.
HOW TO PLAY	Opens the tutorial panel that explains the controls: A/D or the left and right arrows to change lane, W or Space to jump, S to slide, and the equivalent swipe gestures on a touch screen.
CHARACTERS	Opens the character selection screen built from CharacterCatalog. Spark is free, Ember costs 80 coins and Nova costs 160 coins. A locked character is unlocked if the player has enough coins banked.
MISSIONS	Shows the three missions and how far the player has progressed in each: Coin Rush (collect 40 coins), Distance Ace (run 250 metres) and Survivor (stay alive 45 seconds). Each completed mission awards 40 coins.
SOUND ON/OFF	Toggles SaveSystem.Data.soundOn. When sound is turned off, AudioManager stops the music source and every call to Play() returns immediately, so no effect is heard.
QUIT	Calls Application.Quit() to close the build. In the Editor this has no visible effect, which is normal Unity behaviour.

17.3 How the Menu Was Built (Step by Step)

1. Choose GameObject → UI → Canvas. Unity creates the Canvas and, if it does not already exist, an EventSystem as well.
2. On the Canvas, set Render Mode to Screen Space – Overlay so that the UI is drawn on top of the 3D background.
3. On the Canvas Scaler, set UI Scale Mode to Scale With Screen Size and Reference Resolution to 1080 × 1920, so the menu keeps the same proportions on every phone and monitor.
4. Add the title: GameObject → UI → Text – TextMeshPro. Type “Rush Lanes”, set the font size large, choose the cyan colour, and anchor it to the top centre of the screen.
5. Add three more text objects for the statistics row and place them side by side under the title. Name them BestLabel, BankLabel and RunnerLabel.
6. Add the panel: GameObject → UI → Panel. Set its colour to the teal used in the screenshots and resize it so it covers the centre of the screen.
7. Attach a Vertical Layout Group to the panel and set the spacing and padding so the buttons are evenly distributed.
8. Add the six buttons with GameObject → UI → Button – TextMeshPro, and rename them PlayButton, HowToPlayButton, CharactersButton, MissionsButton, SoundButton and QuitButton.
9. Change the label of each button and set the text colour to white so it is readable on the teal panel.
10. For every button, open the On Click () list in the Inspector, press +, drag the RushLanes object into the object field, and choose the matching method from GameFlow or GameUI.
11. Press Play and click each button in turn to confirm that the correct screen opens and that the click sound is heard.

In the finished project all eleven of these steps are performed by UiFactory.cs at runtime. Building the UI from code means the menu cannot be broken by an accidental drag in the Editor, and the same factory method is reused for the tutorial, character, mission and game-over screens.

18. Script 1 — AudioManager.cs

AudioManager is the most unusual script in the project because the game ships without a single audio file. Every sound — the coin pickup, the jump, the slide, the crash, the power-up, the button click, the game-over sting and the background music — is calculated as raw sample data when the game starts and turned into an AudioClip in memory. This keeps the build size very small and means no audio asset can ever go missing from the repository.

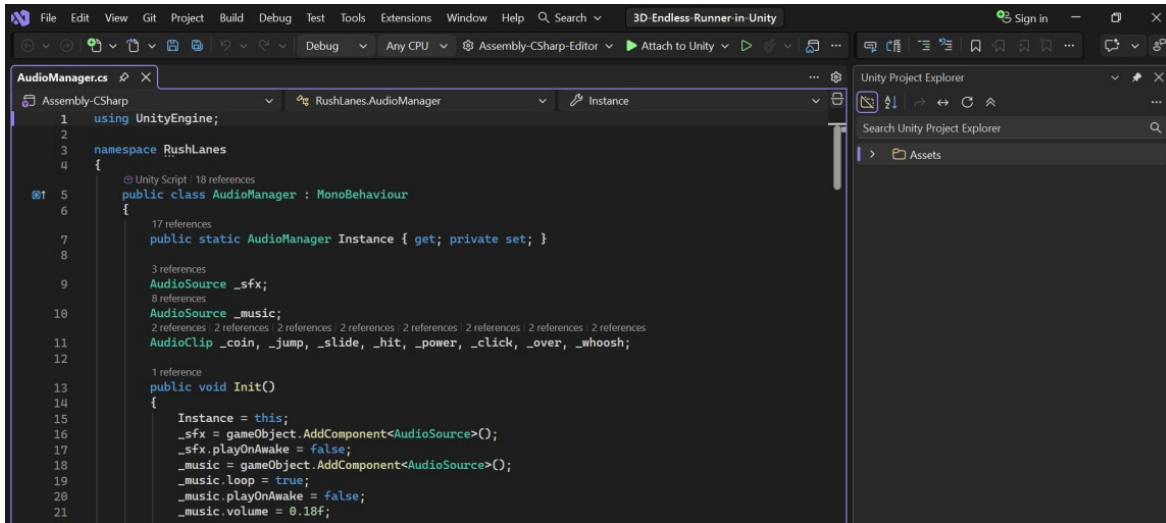


Figure 26: AudioManager.cs — class declaration, singleton Instance and the audio fields.

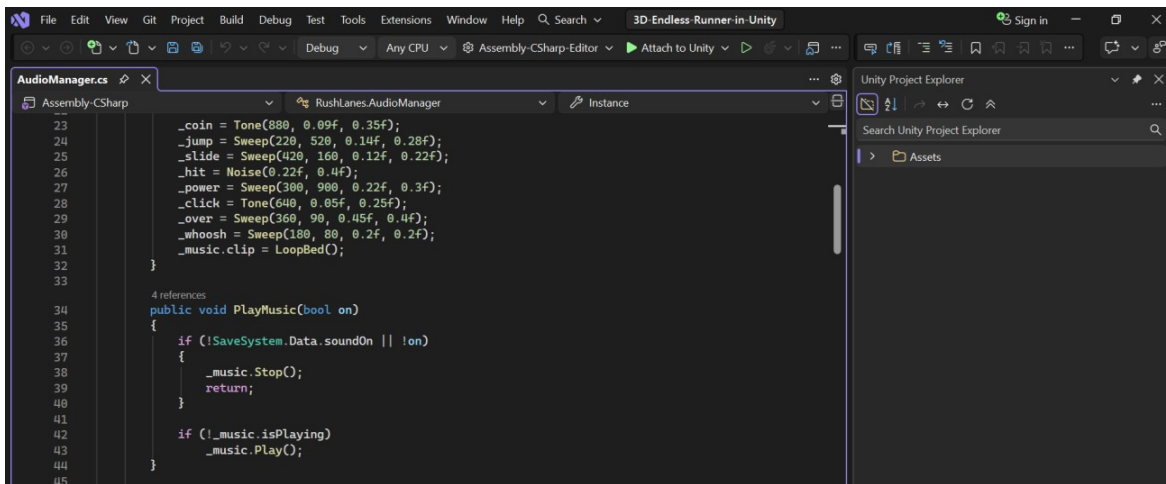


Figure 27: The Init() method building all eight sound effects and the music bed.

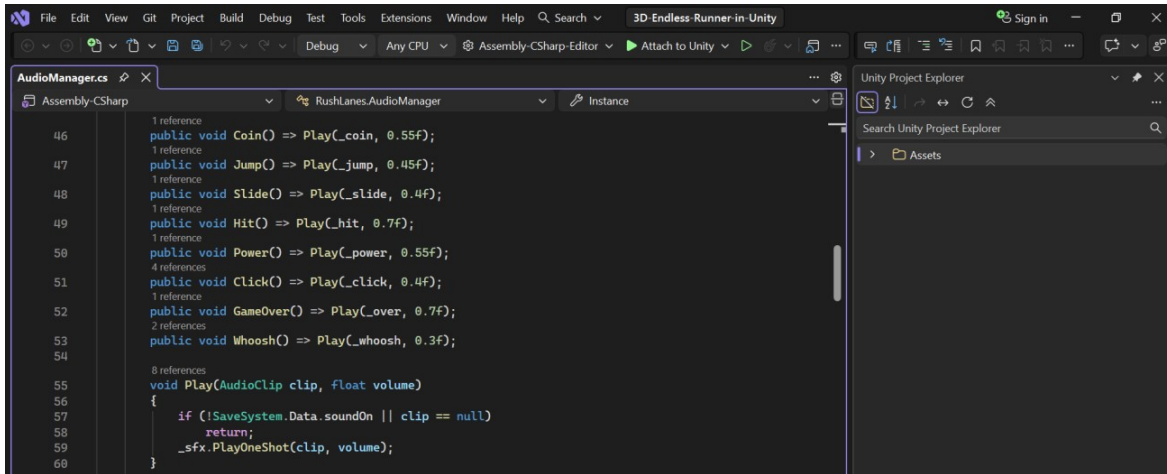


Figure 28: PlayMusic() and the one-line wrapper methods for each sound effect.

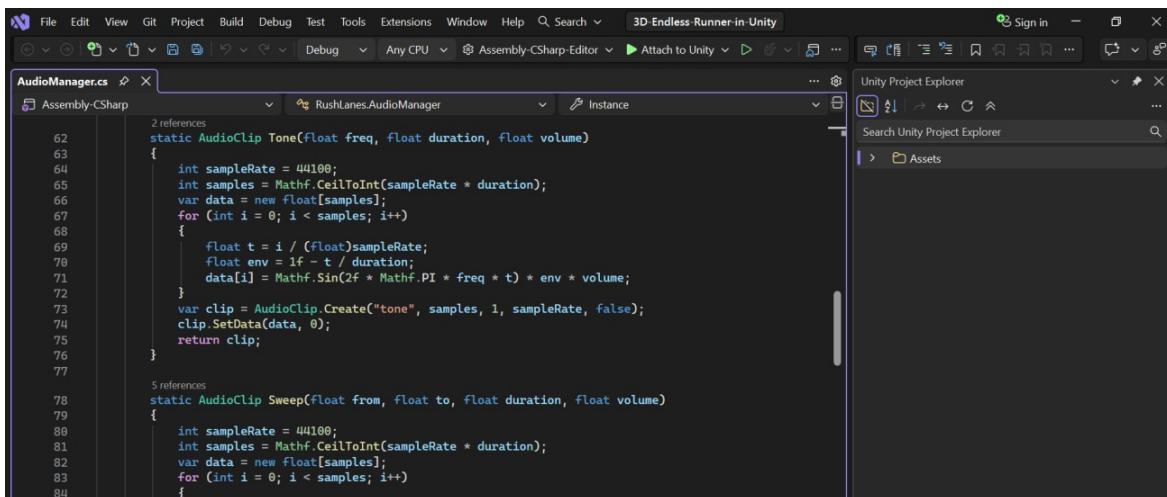


Figure 29: The Tone() and Sweep() generator methods.

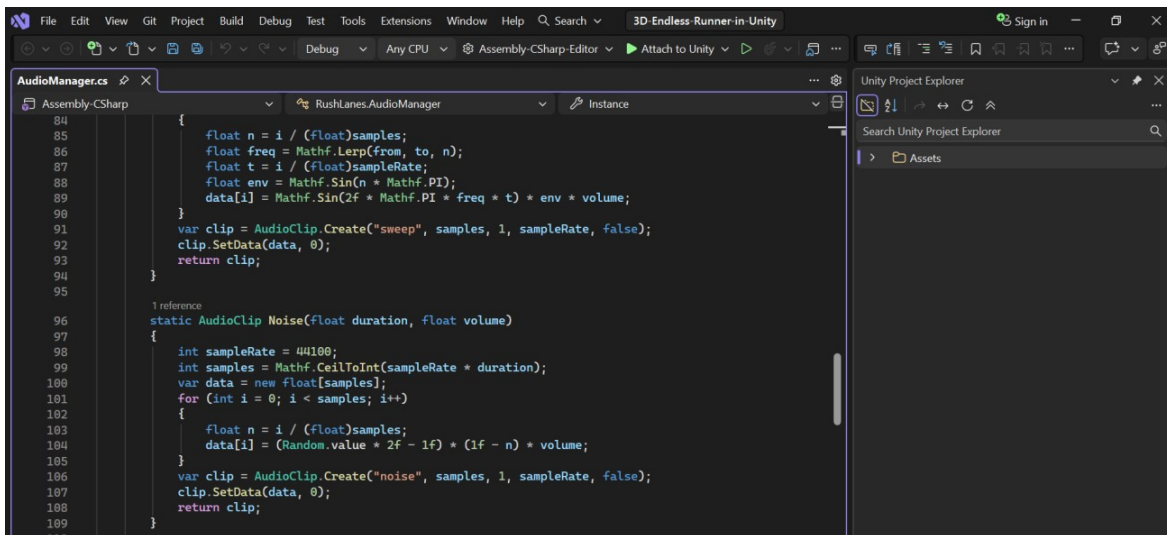


Figure 30: The Noise() generator used for the crash sound.

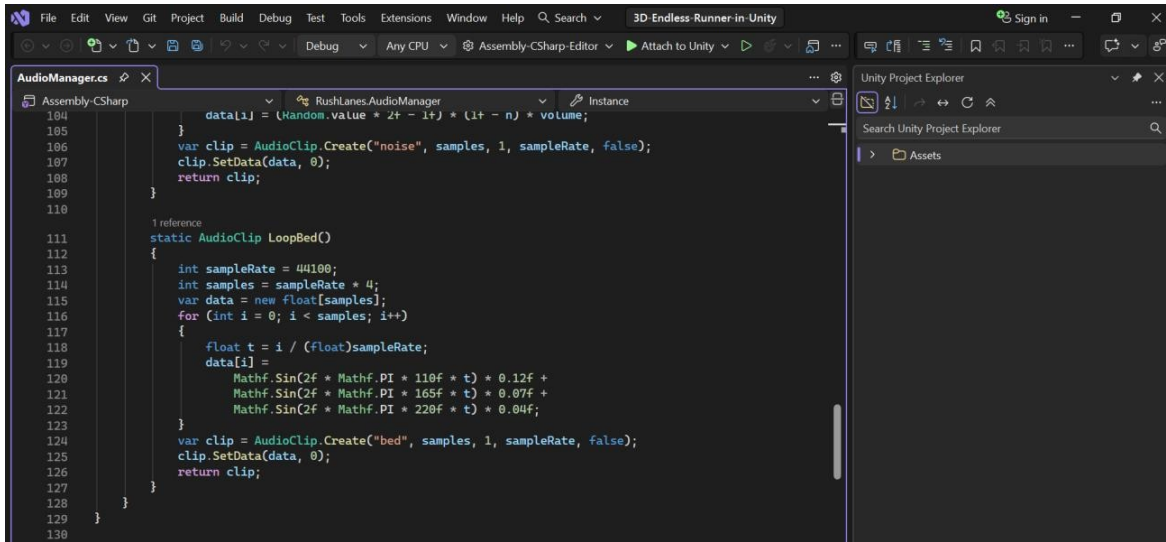


Figure 31: The LoopBed() method that builds the four-second music loop.

18.1 How the Script Is Organised

Member	Signature / Value	Explanation
Instance	public static AudioManager Instance { get; private set; }	A singleton reference so that any script in the game can call AudioManager.Instance.Coin() without holding a direct link to the object.
_sfx / _music	Two AudioSource fields	Added by code in Init(). _sfx plays short one-shot effects, _music loops the background bed at volume 0.18.
Eight AudioClip fields	_coin, _jump, _slide, _hit, _power, _click, _over, _whoosh	Each clip is generated once in Init() and then reused for the whole session, so no memory is allocated during gameplay.
Init()	Called by GameBootstrap	Sets the singleton, creates the two audio sources and fills all eight clips with generated sample data.
PlayMusic(bool)	Start or stop the loop	Checks SaveSystem.Data.soundOn first, so the music respects the SOUND ON/OFF setting from the menu.
Play(clip, volume)	Private helper	A single guarded entry point: if sound is switched off, or the clip is null, it returns immediately without playing anything.

18.2 The Four Sound Generators

Method	What It Generates	Where It Is Used
Tone(freq, duration, volume)	A pure sine wave that fades out linearly	Used for the coin pickup (880 Hz) and the button click (640 Hz). The envelope $1 - t / \text{duration}$ removes the click that would otherwise be heard when the wave stops abruptly.
Sweep(from, to,	A sine wave whose	Used for the jump (220 → 520 Hz, rising), the slide

Method	What It Generates	Where It Is Used
duration, volume)	frequency slides from one value to another	(420 → 160 Hz, falling), the power-up (300 → 900 Hz), the game-over sting (360 → 90 Hz) and the whoosh (180 → 80 Hz). The envelope $\text{Mathf.Sin}(n \times \pi)$ fades the sound in and out smoothly.
Noise(duration, volume)	Random values between -1 and 1 that fade out	Used for the crash sound when the runner hits an obstacle. White noise is the classic way to produce an impact without a recorded sample.
LoopBed()	Three stacked sine waves over four seconds	The background music. 110 Hz at 0.12 amplitude, 165 Hz at 0.07 and 220 Hz at 0.04 are added together, producing a low, calm chord that loops without a seam because four seconds is a whole number of cycles for all three frequencies.

18.3 How a Clip Is Built

All four generators follow the same three steps. The sample rate is 44,100 samples per second — CD quality — so a clip of 0.09 seconds contains 3,969 individual numbers:

1. Work out how many samples are needed: $\text{samples} = \text{Mathf.CeilToInt}(\text{sampleRate} \times \text{duration})$, and create a float array of that size.
2. Fill the array in a loop. For each sample, the time t is calculated, the wave value is computed with Mathf.Sin , the envelope is applied to shape the volume, and the result is multiplied by the requested volume.
3. Create the clip with `AudioClip.Create(name, samples, 1, sampleRate, false)`, copy the array into it with `clip.SetData(data, 0)`, and return it. The third argument, 1, means the clip is mono.

18.4 Complete Source Code

```
using UnityEngine;

namespace RushLanes
{
    public class AudioManager : MonoBehaviour
    {
        public static AudioManager Instance { get; private set; }

        AudioSource _sfx;
        AudioSource _music;
        AudioClip _coin, _jump, _slide, _hit, _power, _click, _over, _whoosh;

        public void Init()
        {
            Instance = this;
            _sfx = gameObject.AddComponent<AudioSource>();
            _sfx.playOnAwake = false;
            _music = gameObject.AddComponent<AudioSource>();
            _music.loop = true;
            _music.playOnAwake = false;
            _music.volume = 0.18f;

            _coin = Tone(880, 0.09f, 0.35f);
            _jump = Sweep(220, 520, 0.14f, 0.28f);
            _slide = Sweep(420, 160, 0.12f, 0.22f);
            _hit = Noise(0.22f, 0.4f);
            _power = Sweep(300, 900, 0.22f, 0.3f);
            _click = Tone(640, 0.05f, 0.25f);
            _over = Sweep(360, 90, 0.45f, 0.4f);
            _whoosh = Sweep(180, 80, 0.2f, 0.2f);
            _music.clip = LoopBed();
        }

        public void PlayMusic(bool on)
        {
            if (!SaveSystem.Data.soundOn || !on)
            {
                _music.Stop();
                return;
            }
            if (!_music.isPlaying)
                _music.Play();
        }

        public void Coin() => Play(_coin, 0.55f);
        public void Jump() => Play(_jump, 0.45f);
        public void Slide() => Play(_slide, 0.4f);
        public void Hit() => Play(_hit, 0.7f);
        public void Power() => Play(_power, 0.55f);
        public void Click() => Play(_click, 0.4f);
        public void GameOver() => Play(_over, 0.7f);
        public void Whoosh() => Play(_whoosh, 0.3f);

        void Play(AudioClip clip, float volume)
        {

```

```

        if (!SaveSystem.Data.soundOn || clip == null)
            return;
        _sfx.PlayOneShot(clip, volume);
    }

    static AudioClip Tone(float freq, float duration, float volume)
    {
        int sampleRate = 44100;
        int samples = Mathf.CeilToInt(sampleRate * duration);
        var data = new float[samples];
        for (int i = 0; i < samples; i++)
        {
            float t = i / (float)sampleRate;
            float env = 1f - t / duration;
            data[i] = Mathf.Sin(2f * Mathf.PI * freq * t) * env * volume;
        }
        var clip = AudioClip.Create("tone", samples, 1, sampleRate, false);
        clip.SetData(data, 0);
        return clip;
    }

    static AudioClip Sweep(float from, float to, float duration, float volume)
    {
        int sampleRate = 44100;
        int samples = Mathf.CeilToInt(sampleRate * duration);
        var data = new float[samples];
        for (int i = 0; i < samples; i++)
        {
            float n = i / (float)samples;
            float freq = Mathf.Lerp(from, to, n);
            float t = i / (float)sampleRate;
            float env = Mathf.Sin(n * Mathf.PI);
            data[i] = Mathf.Sin(2f * Mathf.PI * freq * t) * env * volume;
        }
        var clip = AudioClip.Create("sweep", samples, 1, sampleRate, false);
        clip.SetData(data, 0);
        return clip;
    }

    static AudioClip Noise(float duration, float volume)
    {
        int sampleRate = 44100;
        int samples = Mathf.CeilToInt(sampleRate * duration);
        var data = new float[samples];
        for (int i = 0; i < samples; i++)
        {
            float n = i / (float)samples;
            data[i] = (Random.value * 2f - 1f) * (1f - n) * volume;
        }
        var clip = AudioClip.Create("noise", samples, 1, sampleRate, false);
        clip.SetData(data, 0);
        return clip;
    }

    static AudioClip LoopBed()
    {
        int sampleRate = 44100;

```

```

        int samples = sampleRate * 4;
        var data = new float[samples];
        for (int i = 0; i < samples; i++)
        {
            float t = i / (float)sampleRate;
            data[i] =
                Mathf.Sin(2f * Mathf.PI * 110f * t) * 0.12f +
                Mathf.Sin(2f * Mathf.PI * 165f * t) * 0.07f +
                Mathf.Sin(2f * Mathf.PI * 220f * t) * 0.04f;
        }
        var clip = AudioClip.Create("bed", samples, 1, sampleRate, false);
        clip.SetData(data, 0);
        return clip;
    }
}

```

19. Script 2 — CharacterCatalog.cs

CharacterCatalog defines the three playable runners and the rules for unlocking them. It contains no MonoBehaviour and no scene object — it is a plain static class, which means the character data is available from anywhere in the project without an Inspector reference.

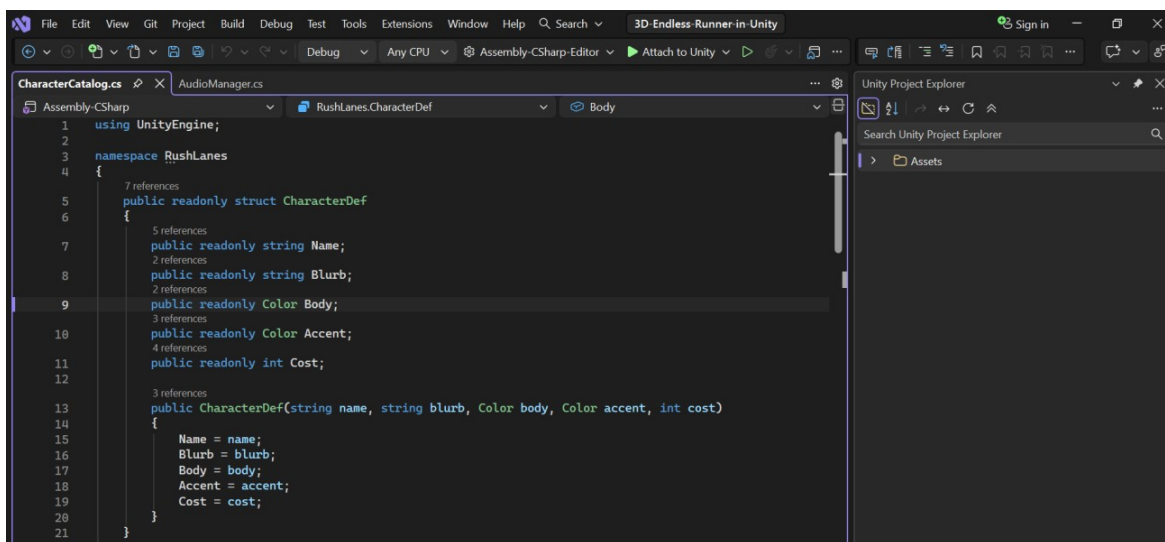


Figure 32: CharacterCatalog.cs — the CharacterDef structure and its constructor.

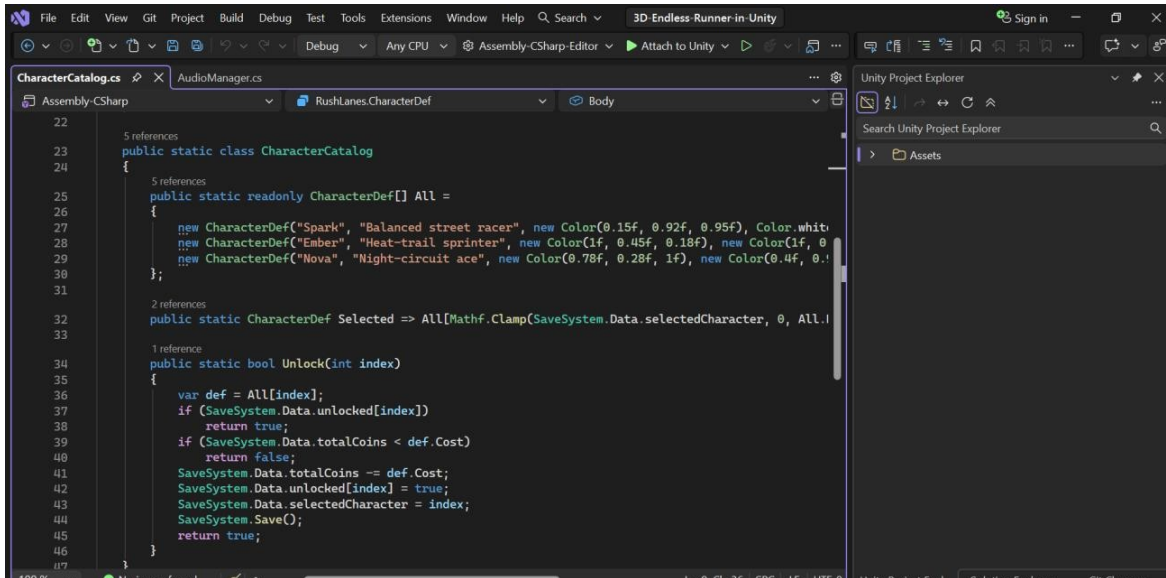


Figure 33: The All array, the Selected property and the Unlock() method.

19.1 The CharacterDef Structure

CharacterDef is declared as a readonly struct. A struct is used instead of a class because the data is small and never changes once it is created, so it can be stored on the stack without allocating memory on the heap. Every field is readonly, which guarantees that a character definition can never be modified accidentally at runtime.

Field	Type	Meaning
Name	string	The display name shown on the character card, for example “Spark”.
Blurb	string	A short one-line description, for example “Balanced street racer”.
Body	Color	The main colour applied to the capsule body of the runner.
Accent	Color	The secondary colour used for the trail and the highlight details.
Cost	int	The number of coins needed to unlock the character. Zero means the character is free.

19.2 The Three Runners

Character	Blurb	Colours and Cost
Spark	Balanced street racer	Cyan body (0.15, 0.92, 0.95) with a white accent · Cost 0 — unlocked from the start
Ember	Heat-trail sprinter	Orange body (1, 0.45, 0.18) with a warm yellow accent (1, 0.85, 0.3) · Cost 80 coins
Nova	Night-circuit ace	Purple body (0.78, 0.28, 1) with a light blue accent (0.4, 0.9, 1) · Cost 160 coins

19.3 Selected and Unlock()

The `Selected` property returns the character the player is currently using. It reads the saved index from `SaveSystem` and passes it through `Mathf.Clamp`, so even if the save file contains an invalid number — for example because a character was removed from the game — the code returns a valid entry instead of throwing an index-out-of-range exception.

`Unlock(index)` performs the purchase and returns true only if the character is available afterwards. The order of the checks matters:

1. If the character is already unlocked, return true immediately — the player is not charged twice.
2. If the banked coins are less than the cost, return false and change nothing.
3. Otherwise subtract the cost from the coin bank, mark the character as unlocked, set it as the selected character, and call `SaveSystem.Save()` so the purchase survives a restart.

19.4 Complete Source Code

```
using UnityEngine;

namespace RushLanes
{
    public readonly struct CharacterDef
    {
        public readonly string Name;
        public readonly string Blurb;
        public readonly Color Body;
        public readonly Color Accent;
        public readonly int Cost;

        public CharacterDef(string name, string blurb, Color body, Color accent, int
cost)
        {
            Name = name;
            Blurb = blurb;
            Body = body;
            Accent = accent;
            Cost = cost;
        }
    }

    public static class CharacterCatalog
    {
        public static readonly CharacterDef[] All =
        {
            new CharacterDef("Spark", "Balanced street racer",
                new Color(0.15f, 0.92f, 0.95f), Color.white, 0),
            new CharacterDef("Ember", "Heat-trail sprinter",
                new Color(1f, 0.45f, 0.18f), new Color(1f, 0.85f, 0.3f), 80),
            new CharacterDef("Nova", "Night-circuit ace",
                new Color(0.78f, 0.28f, 1f), new Color(0.4f, 0.9f, 1f), 160)
        };

        public static CharacterDef Selected =>
            All[Mathf.Clamp(SaveSystem.Data.selectedCharacter, 0, All.Length - 1)];

        public static bool Unlock(int index)
        {
            var def = All[index];
            if (SaveSystem.Data.unlocked[index])
                return true;
            if (SaveSystem.Data.totalCoins < def.Cost)
                return false;
            SaveSystem.Data.totalCoins -= def.Cost;
            SaveSystem.Data.unlocked[index] = true;
            SaveSystem.Data.selectedCharacter = index;
            SaveSystem.Save();
            return true;
        }
    }
}
```

20. Gameplay Systems

With the menu working, the remaining systems complete the game loop. The tables below list the final tuned values that were used in the build.

20.1 Movement

Setting	Value	Reason
Lanes	$x = -2.4 \cdot 0 \cdot +2.4$	Three fixed lanes. The runner moves between them by interpolating towards the target x position, which gives a smooth slide instead of an instant jump.
Rigidbody	Kinematic	A kinematic Rigidbody is used so that movement is fully controlled by the script and Unity's physics cannot push the runner off the track, while collision events are still received.
Speed	Ramps from 10 to 22 m/s	The run starts slow enough to learn the controls and becomes steadily harder, which is what keeps an endless runner interesting.
Jump	Custom gravity of 24	A gravity value much higher than the real-world 9.81 makes the jump short and snappy, so the runner lands quickly and the game stays responsive.
Slide	0.55 seconds	Long enough to pass under a slide gate, short enough that it cannot be used to avoid every obstacle.

20.2 Controls

Action	Input
Change lane	A / D keys, Left / Right arrow keys, or a horizontal swipe
Jump	W key, Up arrow, Space bar, or an upward swipe
Slide	S key, Down arrow, or a downward swipe
Pause / back	Escape key — pauses during a run, goes back one screen in the menus, resumes from the pause panel
UI-tap blocking	SwipeInput ignores any touch that starts on top of a UI element, so tapping a button never makes the runner jump

20.3 Obstacles, Coins and Power-ups

Element	Value	Behaviour
Block	Full-height wall in one lane	Must be avoided by changing lane.
JumpBar	Low bar across a lane	Must be jumped over.
SlideGate	High gate across a lane	Must be slid under.

Element	Value	Behaviour
Difficulty	Grows with distance	As the run continues, obstacles spawn closer together, and at high difficulty two obstacles appear at the same time — but never in all three lanes, so a safe path always exists.
Coins	Spinning rows along the track	Spawned in short rows so the player is rewarded for choosing the harder lane. The magnet power-up pulls nearby coins towards the runner.
Shield	8 seconds	Absorbs one obstacle hit instead of ending the run.
Magnet	8 seconds	Attracts every coin within range.
Boost	4 seconds	Increases speed sharply and passes straight through obstacles.

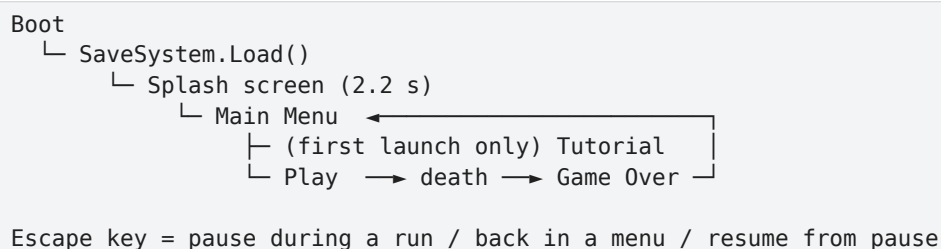
20.4 Missions and Saved Data

Mission	Target	Reward
Coin Rush	Collect 40 coins in one run	+40 coins
Distance Ace	Reach 250 metres in one run	+40 coins
Survivor	Stay alive for 45 seconds	+40 coins

SaveSystem stores the following values as JSON inside PlayerPrefs, so progress is kept between sessions: the high score, the best distance, the total coins banked, which characters have been unlocked, the currently selected character, the sound on/off setting, and the progress of each mission.

20.5 Game Flow

The complete state machine implemented in GameFlow.cs runs as follows:



21. Conclusion

Part B of this internship task extended the basic Unity scene from Part A into a complete, playable endless runner. The main screen now contains a working Event System, a 3D background made of a scaled ground plane and a capsule runner, an animated menu preview, and a persistent manager object that carries the

audio, UI and game-flow systems through every scene change. The menu itself offers six options, and every one of them leads to a finished screen.

On the scripting side, twenty-one C# scripts and one editor tool were written inside a single namespace. Two of them are documented in detail in this report: AudioManager, which generates every sound in the game from sine waves and white noise instead of audio files, and CharacterCatalog, which defines three runners and the coin economy that unlocks them. Building the scene and the interface entirely from code proved to be the most valuable lesson of the internship — the project can be cloned from GitHub and run immediately on any machine, with no missing references and no broken asset links.

Repository

Full source code: <https://github.com/gasimzahoor825/Rush-Lanes>